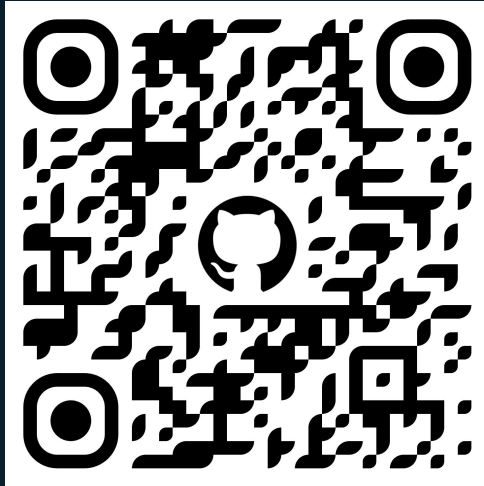


Resource



github.com/ro-witthawin/TBS-BI303-2026

TBS-BI303-2026



Class schedule

Date	Topic	Material
2026-08-03	Intro to Programming Language	Day01
2026-08-10	Basic String & List	Day02
2026-08-17	Function	Day03
2026-08-24	Loop	Day04
2026-08-31	Dictionary	Day05
2026-09-07	Class Object	Day06
2026-09-14	Freq Library	Day07
2026-09-21	Final Exam	

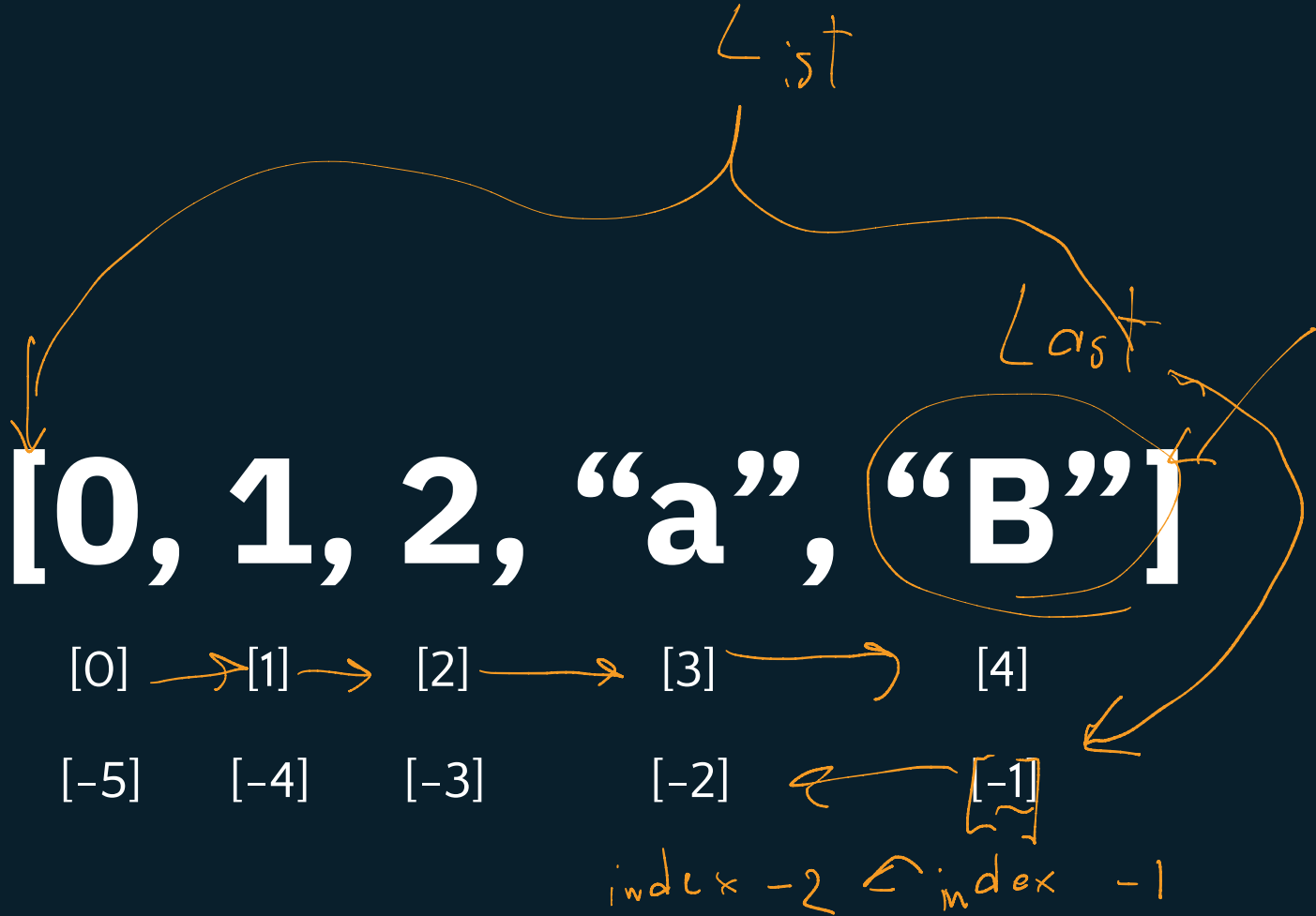
“List”

[...]

List

[0, 1, 2, "a", "B"]

List



Empty List



List indexing

sample = ["apple", "orange", "pen"]

sample[0] # "apple"

sample[-1] # "pen"

sample[0:2] # ["apple", "orange"]

sample[1:] # ["orange", "pen"]

sample[1:-1] # ["orange"]

Human 1 2 3 4 5

Com 0 1 2 3 4

["apple" , 0 , 7.12]
↓ ↓ ↓
str int float

sample[0] → str
[1] → int
[-1] → float

type

[]

List Methods

```
sample = ["apple", "orange", "pen"]
```

sample.append(value) # Add one item to the end

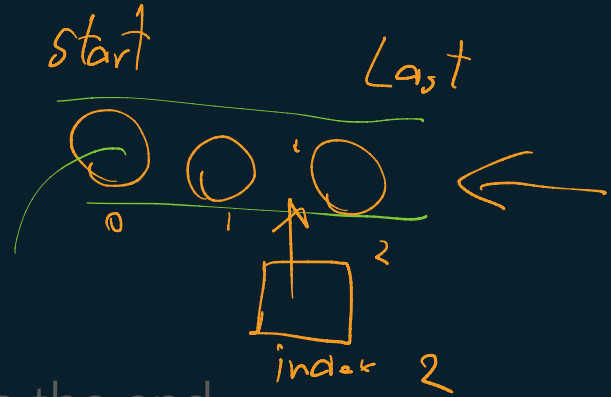
sample.insert(index, value) # Add one item at a specific position

sample.extend(list) # Add multiple items from another list

sample.remove(value) # Remove a specific value

sample.pop(index) # Remove an item by index and return it

sample.clear() # Remove all items

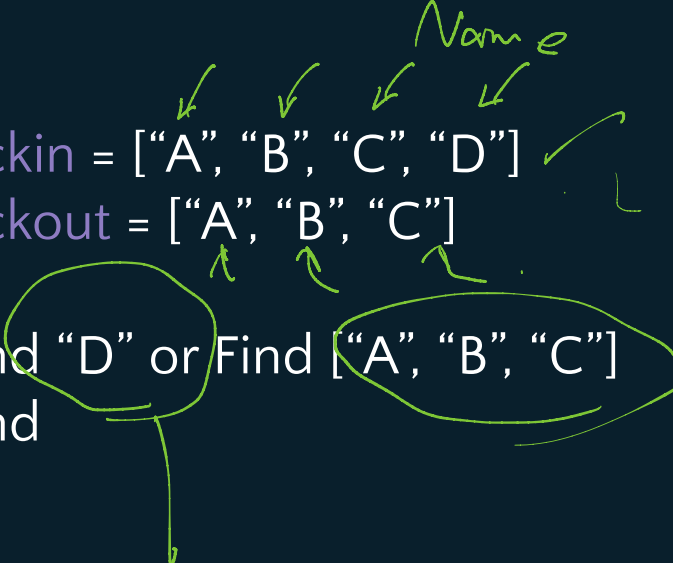


List : Example Case #1 Attendance Check

Name

✓ checkin = ["A", "B", "C", "D"]
✓ checkout = ["A", "B", "C"]

Find "D" or Find ["A", "B", "C"]
Find



List : Example Case #2 Attendance Check

checkin = [

→ ["A", "09:00"], ←

→ ["B", "09:01"], ←

→ ["A", "09:03"] ←

]

List 2D

Find average of "A"

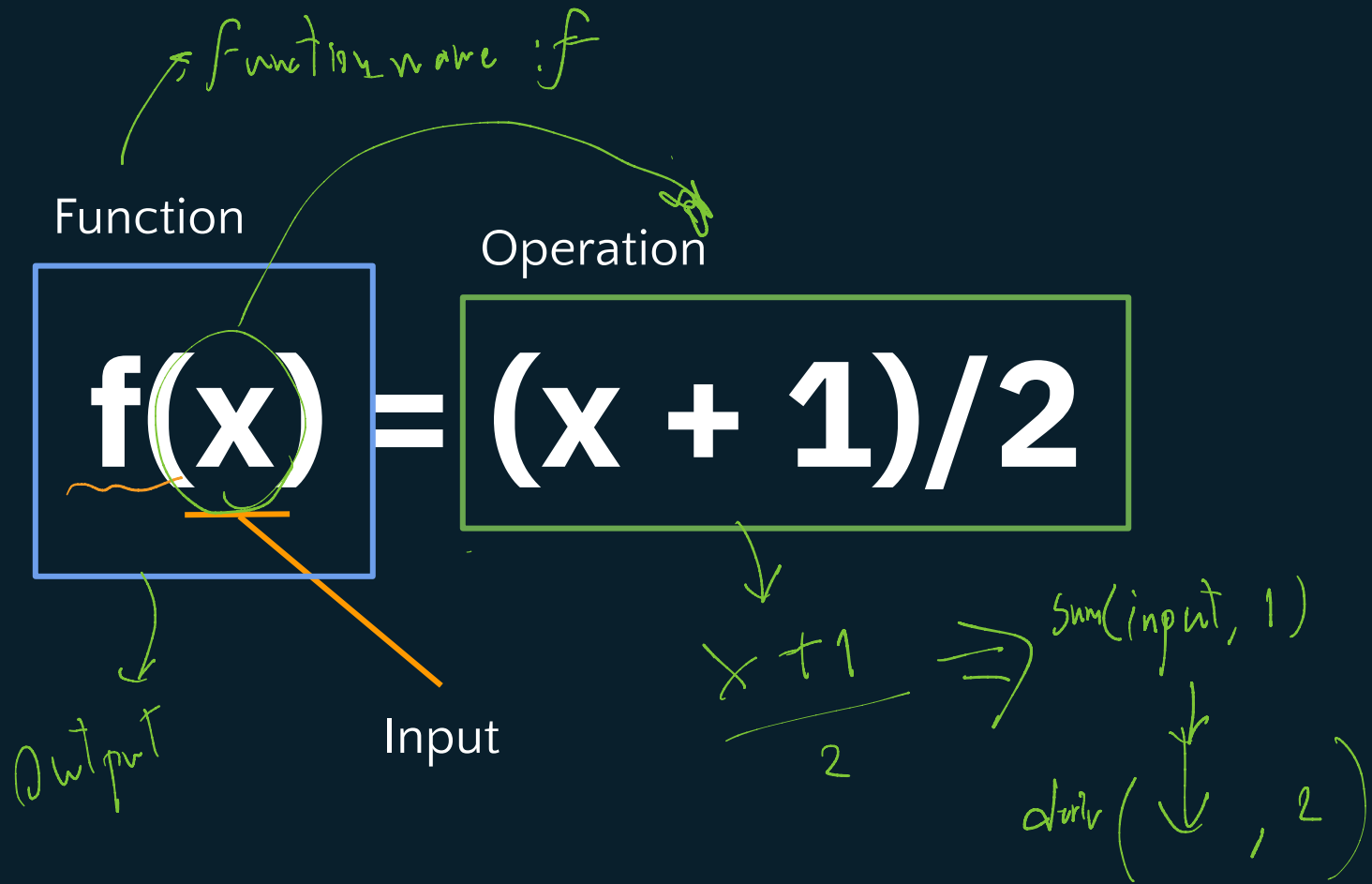
List 2D → Table

	Name	Time	
0	"A"	"09:00"	✓
1	"B"	"09:01"	✓
2			✓
3			
4			

Table → List 2D

“Function”

$f(x)$



Function in Math

Equation / Function

$$v = v_0 + a * t^2$$

velocity

Power

human

speed = init_speed + accelerate * time**2

v

v_0

$v_{t=0}$

a

t

Function in Math

$$v = v_0 + a * t^2$$

Equation

$$\text{speed} = \text{init_speed} + \text{accelerate} * \text{time}^{**2}$$

Human

Function

Function name

Input

Operation

Function

The diagram shows a function call `speed(init_speed, accelerate, time)` enclosed in a blue box. The word "Function" is written below the box. The word "Function name" is written in green below the `speed` parameter. The word "Input" is written below the three parameters. The entire function call is circled in orange. To the right, the expression `= init_speed + (accelerate * (time**2))` is shown. The word "Operation" is written in green below the `+` operator. The word "Function" is written in green above the `(time**2)` sub-expression. The entire expression is circled in white.

Function in Python

$$v_t = v_0 + a * t^2$$

Equation

Human

speed(init_speed, accelerate, time) = init_speed + accelerate * time**2

```
def speed(init_speed, accelerate, time):  
    → output_value = init_speed + accelerate * time**2  
    → return output_value
```

v_t = speed(0, 9.8, 10) ✓

v_t = speed(init_speed = 0, accelerate = 9.8, time = 10) ✓

$$v_t = 0 + 9.8 * 10^2 = 980$$

Function in Python

$$v_t = v_0 + a * t^2$$

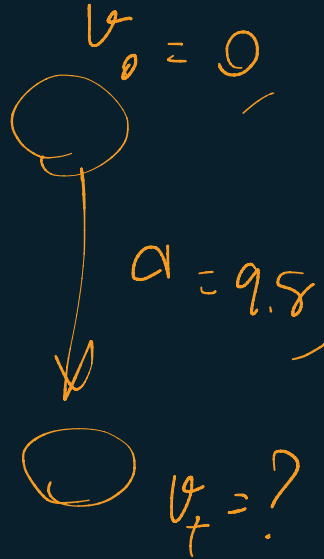
`speed(init_speed, accelerate, time) = init_speed + accelerate * time**2`

```
def speed(init_speed, accelerate, time):  
    output_value = init_speed + accelerate * time**2  
    return output_value
```

`v_t = speed(0, 9.8, 10)` ✓

`v_t = speed(init_speed = 0, accelerate = 9.8, time = 10)` ✓

Can we define as default value?



Function in Python

$$v_t = v_0 + a * t^2$$

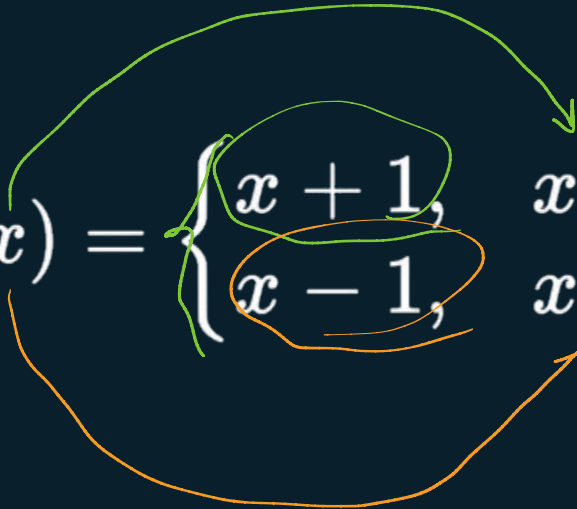
input (argument)

default value

```
def speed(time, init_speed = 0, accelerate = 9.8):  
    output_value = init_speed + accelerate * time**2  
    return output_value
```

- `v_t = speed(10)` ✓ $0 + 9.8 * 10^2 = 980$
- `v_t = speed(10, 9.8)` ✓ # `t=10, v_0=9.8, a=9.8`
- `v_t = speed(10, 0, 9.8)` ✓ # `t=10, v_0=0, a=9.8`
- `v_t = speed(init_speed = 0, accelerate = 9.8, time = 10)` ✓
- `v_t = speed(time = 10, init_speed = 0, accelerate = 9.8)` ✓

Function Condition

$$f(x) = \begin{cases} x + 1, & x > 0 \\ x - 1, & x < 0 \end{cases}$$
The image shows the piecewise function definition with several hand-drawn annotations. A green arrow starts from the expression $x + 1$ and points to the condition $x > 0$. An orange arrow starts from the expression $x - 1$ and points to the condition $x < 0$. Additionally, the expression $x + 1$ is circled in green, and the expression $x - 1$ is circled in orange.

Function Condition

$$f(x) = \begin{cases} x + 1, & x > 0 \\ x - 1, & x < 0 \end{cases}$$

```
def eq1(x):  
    if x > 0:  
        output_value = x + 1  
    elif x < 0:  
        output_value = x - 1  
    return output_value
```

```
def eq1(x):  
    if x > 0:  
        return x + 1  
    elif x < 0:  
        return x - 1
```

Hand-on 10 min without AI

Car. Velocity



$$v(t) = \begin{cases} v_{max}, & v_t \geq v_{max} \\ v_0 + at^2, & \text{other} \end{cases}$$

Handwritten notes: 120 km/h

```
def speed(...):
```

```
·  
·  
·
```

```
return ...
```

Settings

Site > ☒ Show AI-powered inline completions

Editor > ☒ Consented to use generative AI features

AI assistance > ☒ Hide generative AI features

Colab Pro > Gemini can make mistakes, so double-check responses and [use code with caution](#).

GitHub >

Miscellaneous >

Level : ★★★★★

$$\underline{v(t)} = \begin{cases} \underline{v_{max}} & ; \textcircled{v(t)} > v_{max} \\ \underline{v_0 + at^2} & ; \text{other} \end{cases}$$

output

Logic

120

return v_t
(output)

output v_t $\begin{cases} v_t \\ v_{max} = 120 \end{cases}$

Input ($t, v_0, a, \underline{v_{max}}$)

$$\underline{v_t} = v_0 + (a * t^2)$$

if ($v_t \geq v_{max}$):
return v_{max}
else
return v_t

Function is more than just Math

```
def imageGenerate(prompt):
```

```
    new_prompt = explainPrompt(prompts)
```

```
    init_image = text2image(new_prompt)
```

```
    reconstruct_image = diffusion(init_image)
```

```
    final_image = upscale(reconstruct_image)
```

```
    return final_image
```

def explainPrompt

def text2image

def diffusion

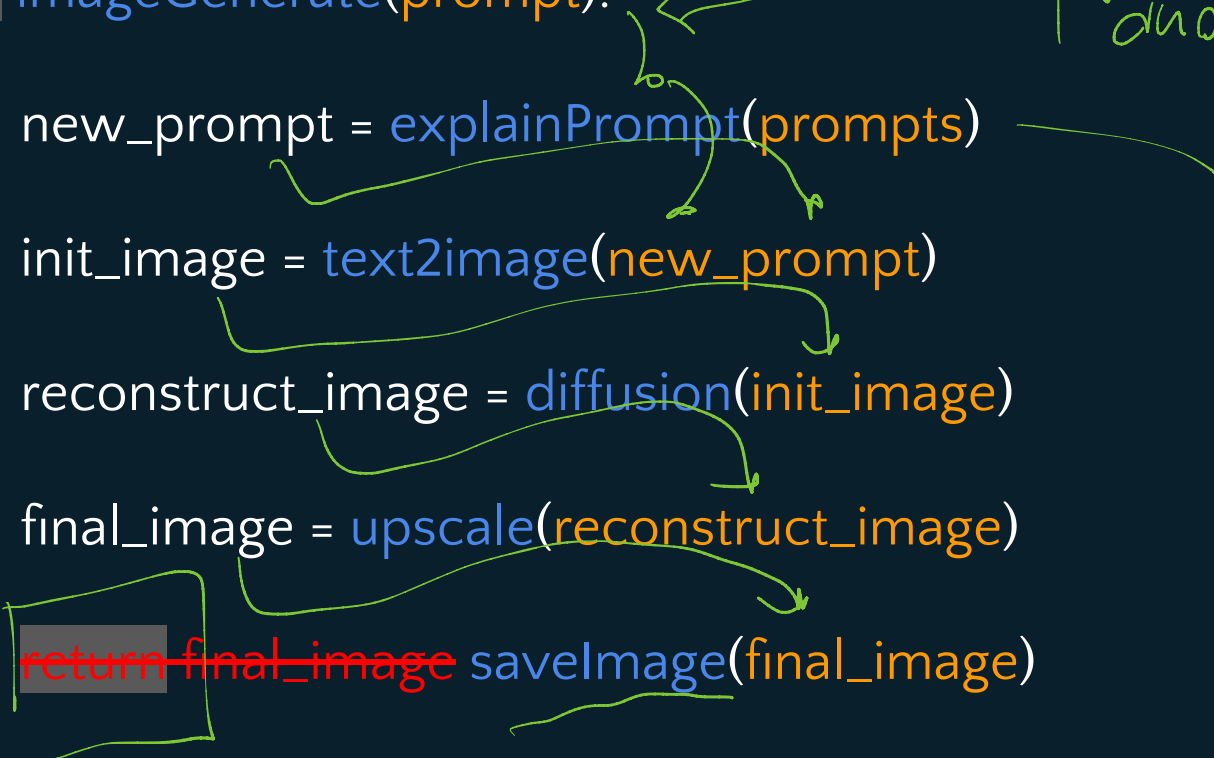
print (

str

imageGenerate(

Sometime we don't need output

```
def imageGenerate(prompt):  
    new_prompt = explainPrompt(prompts)  
    init_image = text2image(new_prompt)  
    reconstruct_image = diffusion(init_image)  
    final_image = upscale(reconstruct_image)  
    return final_image saveImage(final_image)
```



Nano Bernano

Sometime function can return more

```
def splitFullName(fullname): # Expect like "Witthawin S"
    name = fullname.lower().strip() # "witthawin s"
    first_space_index = name.find(" ") # 9
    space_count = name.count(" ") # 3
    firstname = name[:first_space_index] # "witthawin"[:9]
    lastname = name[first_space_index+space_count:] # "s"[:12:]
    return firstname, lastname
```

Handwritten annotations:

- Red curly braces above "Witthawin" and "S" in the comment.
- Red circle around the space character in the `find` method.
- Orange circle around the number 9 in the `find` result.
- Orange arrow from the space character in the `count` method to the number 3.
- Orange arrow from the space character in the `count` method to the `count` method call.
- Orange arrow from the `first_space_index` variable to the slice `[:first_space_index]`.
- Orange arrow from the `first_space_index+space_count` expression to the slice `[first_space_index+space_count:]`.
- Orange arrow from the `firstname` variable to the `return` statement.
- Orange arrow from the `lastname` variable to the `return` statement.
- Handwritten text: `count(" ")` next to the `space_count` line.
- Handwritten text: `9 + 3` next to the `lastname` slice.

```
firstname, lastname = splitFullName("Witthawin S.")
# firstname = "witthawin", lastname = "s."
```

Argument setup

```
def function_name(input_1: str,
                  input_2: int,
                  input_3: float,
                  input_4: str,
                  input_5: list,
                  input_6 = None,
                  input_7: int | None = None
):
    pass # do-something
```

$$\underline{v}_+ = \underline{v}_0 + \underline{a} t^2$$

Next Week

None Type

None

Any Function without return any value, it will return None

Function without return

```
def imageGenerate(prompt: str):  
  
    new_prompt = explainPrompt(prompts)  
    init_image = text2image(new_prompt)  
    reconstruct_image = diffusion(init_image)  
    final_image = upscale(reconstruct_image)  
    saveImage(final_image)  
  
imageGenerate("Bird with Pen") # Return None
```

Function without return

$$f(x) = \begin{cases} x + 1, & x > 0 \\ x - 1, & x < 0 \end{cases}$$

```
def eq1(x):  
    if x > 0:  
        return x + 1  
    elif x < 0:  
        return x - 1
```

```
y = eq1(0) # y= None
```

```
def eq1(x):  
    if x > 0:  
        output_value = x + 1  
    elif x < 0:  
        output_value = x - 1  
    return output_value
```

```
y = eq1(0) # Error
```

How to check None

x is None

x is not None

How to check None: Example



```
def purchasing Iphone(income=None):  
    if income is None:  
        return False  
  
    if income > 30000:  
        return True  
    else:  
        return False
```

Caution of Function

```
def func_1(x=None):  
    a = 0  
    b = 1
```

```
def func_2(x=None):  
    a = 1  
    b = 2
```

`a` in `func_1` is distinct
from `a` in `func_2`.

```
def func_1(x=None):  
    a = 0  
    b = 1
```

```
def func_2(x=None):  
    a = 1  
    b = 2
```

`func_2()` ✓

`func_1()` ✓

`func_2()` ✗

Hand-on 20 min without AI

```
def iphone_plan(  
    iphone_price: float,  
    monthly_income: float,  
    Installment_months: int | None = None):
```

```
    pass # do_something
```

```
    return n_month, monthly_payment, status
```

```
# status = "Affordable" or "Insufficient"
```

```
# months_needed : int, monthly_payment : float
```



Level : ★★☆☆☆